

# PocketLLM Portal: Architectural Implementation Report

Team Project - CSCI 578 Software Architectures

## Executive Summary

PocketLLM Portal is a lightweight, privacy-friendly assistant designed to run on consumer hardware, allowing users to upload documents and receive grounded answers via a local Large Language Model (LLM). In Assignment 3, the system was prescribed as a React Single-Page Application (SPA) communicating with a polyglot backend (Python/Legacy services).

The final implementation realizes this vision but introduces a significant architectural shift to a unified MERN stack (MongoDB, Express.js, React, Node.js). This report analyzes the “Descriptive Architecture” (what was built) in relation to the “Prescriptive Architecture” (Assignment 3 design). While the core functional requirements for chat, session management, and document analysis are met, the backend architecture exhibits intentional **Architectural Drift**, moving from Python to Node.js to streamline development, and minor **Architectural Erosion** in the telemetry and vector search subsystems.

## 1. Architecture Selection Rationale

### 1.1 Evaluation Criteria

Six architectural candidates were evaluated on:

1. **Novelty & Creativity:** Unique features, privacy-preserving approaches, innovative LLM integration
2. **Architecture Quality:** Modularity, scalability, technical feasibility, clear boundaries
3. **Model Selection:** Hardware appropriateness, integration ease, performance, community support

### 1.2 Selection

**Winner:** React SPA design(Hitesh’s architecture) was selected for:

### Novelty:

- React SPA with concrete components (AppShell, Router, AuthContext, ApiClient, ErrorBoundary)
- RAG as first-class feature with document assistant
- Client-side routing (ChatPage, SearchPage, HistoryPage, AdminPage, DocsPage)
- Multi-tier caching (IndexedDB → Memory → DB)
- Privacy-first: 100% local execution

### Quality:

- Three-layer architecture: Presentation (React) → Application (Services) → Data (MongoDB + Ollama)
- Component-based frontend with clear separation of concerns
- APIClient pattern isolating HTTP from UI
- RESTful API with JWT authentication
- Docker-based deployment

### Model Selection:

- Ollama framework for optimized CPU inference (4-bit quantization)
- Llama 2 7B Chat (~4GB RAM, runs on target hardware)
- HTTP API enabling language-agnostic integration

### 1.3 Capability Subset Determination

Implemented (80% of Hitesh vision)

| Requirement                    | Description                                         | Status      | Rationale                                              |
|--------------------------------|-----------------------------------------------------|-------------|--------------------------------------------------------|
| <b>FE1: Chat Interface</b>     | Real-time chat with streaming responses and history | <b>Full</b> | Core functionality; validates Ollama + SSE streaming   |
| <b>FE2: Session Management</b> | Create, resume, rename, and delete chat sessions    | <b>Full</b> | Essential for workflows; validates MongoDB persistence |

|                                    |                                                             |                |                                                      |
|------------------------------------|-------------------------------------------------------------|----------------|------------------------------------------------------|
| <b>FE3: Document Upload</b>        | Ingestion of text/PDFs for context retrieval                | <b>Full</b>    | Differentiator from basic chatbots; enables RAG      |
| <b>FE4: Grounded Answers (RAG)</b> | Retrieval-Augmented Generation (RAG) using uploaded content | <b>TF-ID F</b> | End-to-end pipeline works; simplified from vector DB |
| <b>FE5: Admin Console</b>          | System configuration and usage metrics                      | <b>Full</b>    | Multi-user config; demonstrates RBAC                 |

#### Descoped:

- FE6: DocsPage UI → Markdown docs in repository (better for developers)
- SearchPage → Integrated into ChatInterface (improved UX)
- ErrorBoundary → Simplified error handling (lower priority)

#### NFRs Satisfied:

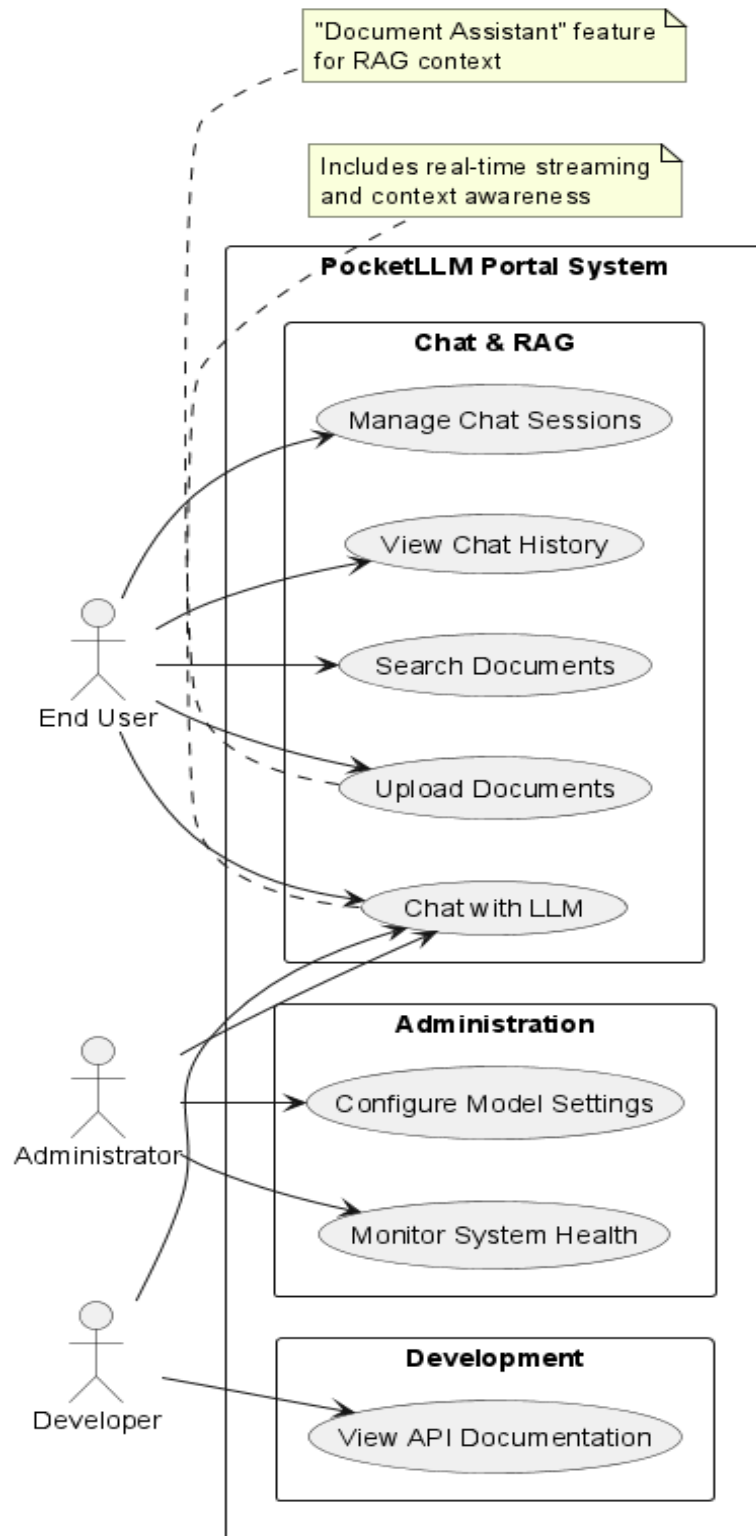
- NFR1 (Hardware): Runs on standard 16GB RAM, CPU-only hardware
- NFR2 (Latency): Fast UI navigation and acceptable inference speed
- NFR3 (Throughput): Concurrent users
- NFR4 (Availability): 100% offline
- NFR5 (Privacy): Zero external calls. All data and inference must remain local.
- NFR6 (Security): JWT + RBAC
- NFR7 (Maintainability): Docker + docs

## 2. UML Models

The following diagrams illustrate the contrast between the prescribed design and the final implemented structure.

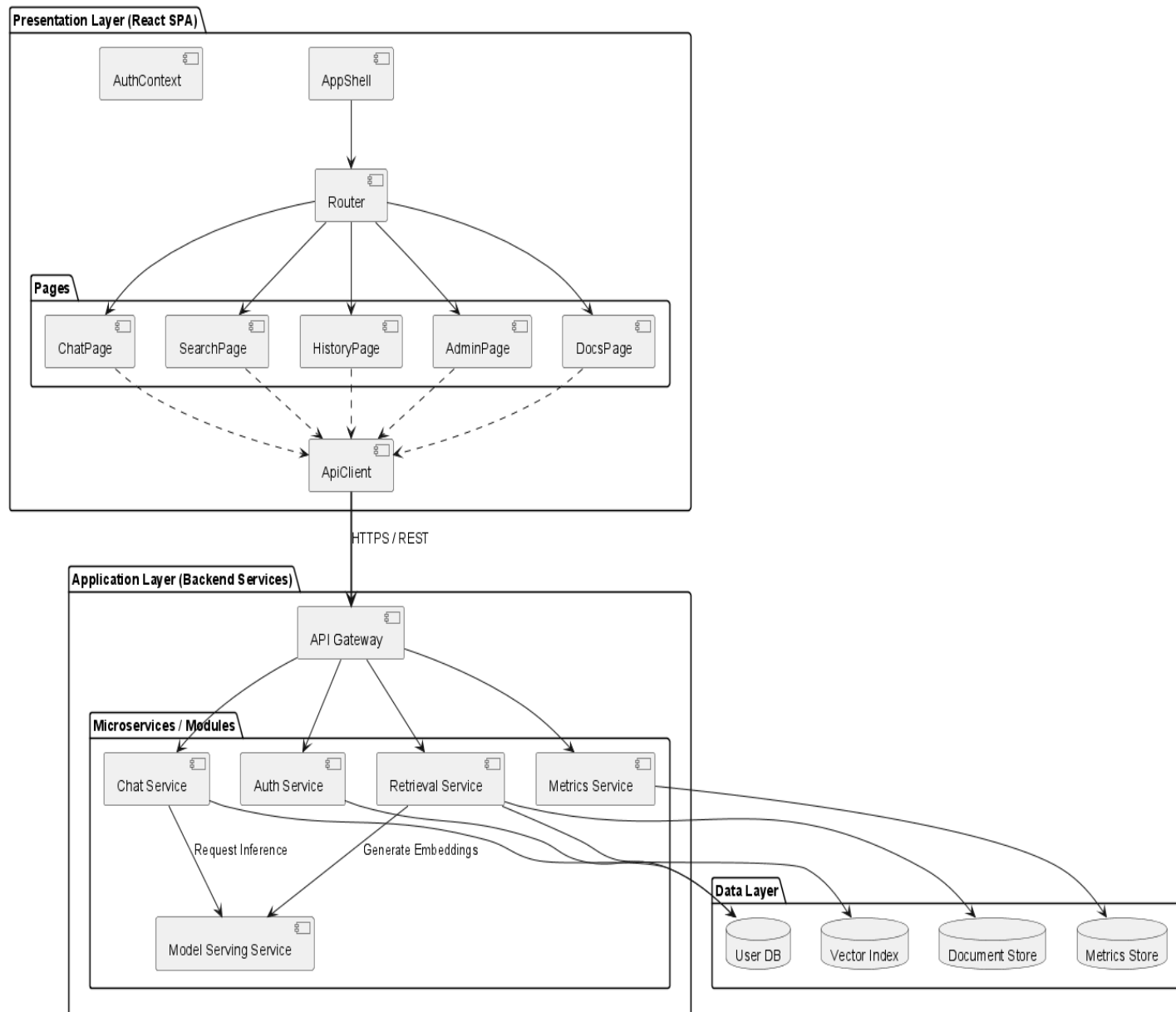
#### Figure 1. Use-Case Diagram (Prescriptive)

The actors (End User, Admin, Developer) and their goals remain unchanged. The system boundary encapsulates the React SPA and the local backend services.



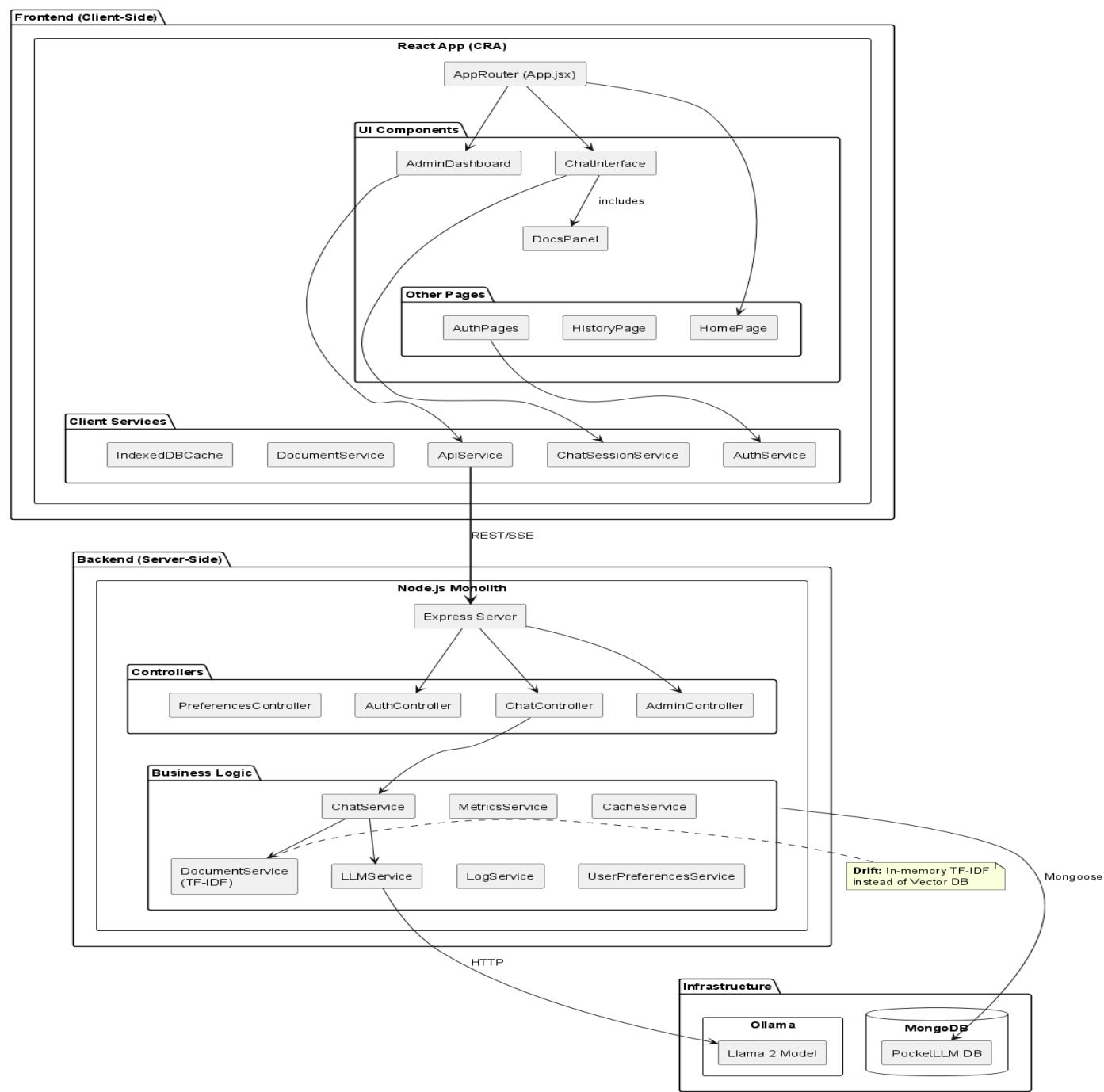
**Figure 2. Component Diagram (Prescriptive)**

The original design separated the frontend (React) from a set of abstract backend services (Semantic Search, Model Serving) that could be implemented in Python.



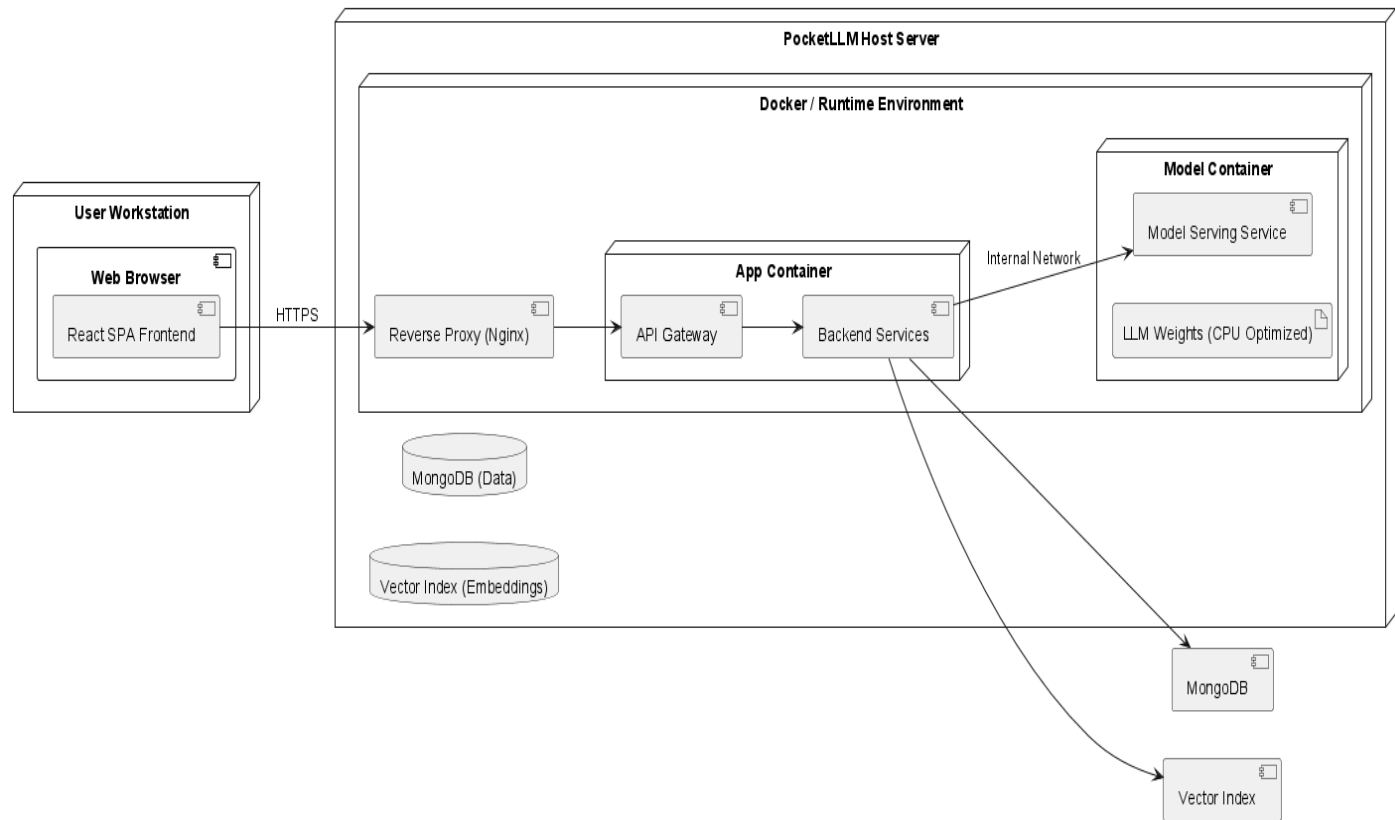
**Figure 3. Component Diagram (Descriptive - Implemented)**

The actual implementation unifies the backend into a Node.js Monolith (server.js) containing modular controllers and services. The “Model Serving” component is realized as an external **Ollama** container. The “Vector Index” is replaced by an in-memory **TF-IDF** search within DocumentService.



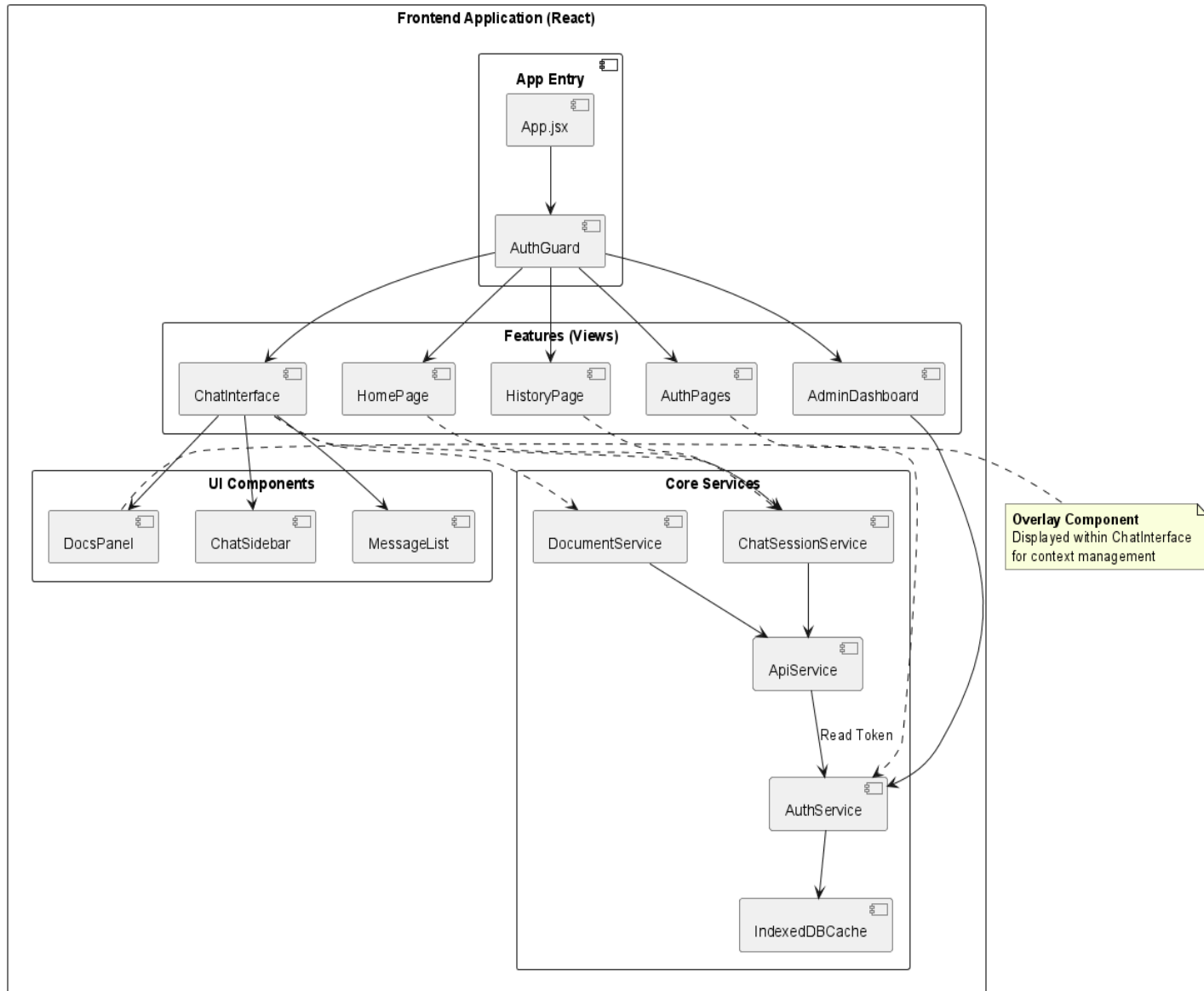
**Figure 4. Deployment Diagram (Prescriptive)**

The deployment view confirms the single-host strategy. The Browser (React bundle) connects via HTTPS to the Application Server, which manages local storage (MongoDB) and the Inference Engine (Ollama).



**Figure 5. Frontend Architecture (Descriptive - Implemented)**

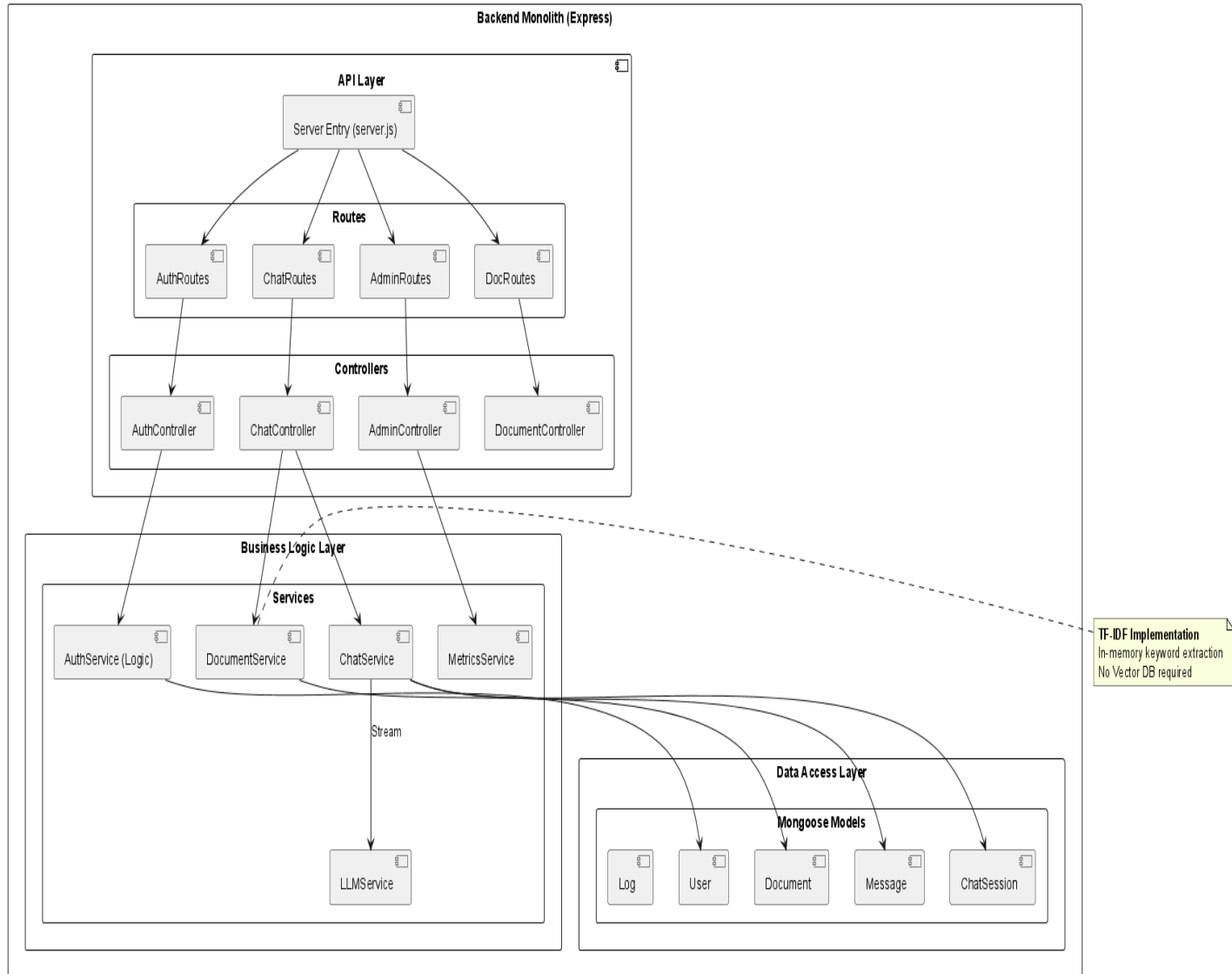
The React frontend implements a component-based architecture with clear separation between presentation (‘ChatInterface’, ‘SessionList’, ‘AdminDashboard’), state management (‘AuthContext’, ‘IndexedDB’), and API integration (‘ApiClient’). The routing layer uses React Router for navigation between chat, history, and admin views.



**Figure 6. Backend Architecture (Descriptive - Implemented)**

The Node.js backend follows a service-oriented pattern within a monolithic structure. Controllers ('chatController', 'authController', 'adminController') handle HTTP routing, while specialized services ('ChatService', 'LLMService', 'DocumentService', 'CacheService') encapsulate business logic. Data persistence is managed through Mongoose models connecting to MongoDB, with the LLMService interfacing with the external Ollama container via HTTP.





### 3. Architectural Drift & Erosion Analysis

#### 3.1 Analysis of Architectural Drift and Erosion

This section details the variances between the Assignment 3 design (Prescriptive) and the Final Implementation (Descriptive).

#### Backend Technology Stack

- **Prescriptive (Design): Polyglot / Python.** Implied separate services for Search/LLM (Python) and API (Generic).

- **Descriptive (Implementation): Node.js Monolith.** Unified Express.js server handling Auth, Chat, and Logic.
- **Classification: Drift** (Intentional)
- **Rationale:** Homogenizing the stack to TypeScript/JavaScript (MERN) reduced context switching and cognitive load for the team.
- **Architectural Impact:** Simplifies build pipeline and deployment (single npm start), but couples services tighter than microservices.

## Inference Engine

- **Prescriptive (Design): Custom Model Service.** “Model Serving Service” component managing weights directly.
- **Descriptive (Implementation): Ollama Container.** External service managing Llama 2 via HTTP API.
- **Classification: Drift** (Optimization)
- **Rationale:** Writing a custom Python inference loader is error-prone and inefficient. Ollama provides a robust, optimized runtime for CPU inference (NFR1).
- **Architectural Impact:** Improves reliability and modularity; the LLM can be swapped without changing backend code.

## Search Algorithm

- **Prescriptive (Design): Vector Database.** Semantic search via embeddings (e.g., Chroma/Faiss).
- **Descriptive (Implementation): TF-IDF / Keyword.** In-memory statistical text retrieval in DocumentService.
- **Classification: Erosion** (Simplification)
- **Rationale:** Setting up a dedicated Vector DB was deemed “over-engineering” for the prototype scale.
- **Architectural Impact:** Retrieval quality is lower (keyword vs. semantic meaning), representing an erosion of capability (FE4), though functional complexity is reduced.

## Telemetry

- **Prescriptive (Design): System Metrics.** CPU/RAM usage, low-level host stats.
- **Descriptive (Implementation): Application Metrics.** Request counts, latency, DB status.
- **Classification: Erosion** (Scope Reduction)
- **Rationale:** Accessing host OS metrics from a containerized Node app requires elevated privileges.
- **Architectural Impact:** Admin has less visibility into hardware saturation (NFR1), relying instead on application throughput as a proxy.

## Documentation

- **Prescriptive (Design): DocsPage UI.** Interactive documentation page in the app.
- **Descriptive (Implementation): Markdown / Repo.** Static API\_DOCUMENTATION.md files.
- **Classification: Drift** (Workflow)
- **Rationale:** Developers prefer markdown in the repo over browsing a UI.
- **Architectural Impact:** Reduces frontend code footprint; documentation is version-controlled alongside code.

## 3.2 Rationale for the MERN Architecture

### Why Node.js over Python?

While Python is standard for AI, the integration requirement for this project was “Consumer Hardware” and “Web Application.” Node.js handles I/O-bound HTTP requests (such as streaming chat tokens) more efficiently than synchronous Python web servers. By offloading the heavy computation to Ollama (Go-based), the Node backend remains lightweight and responsive (NFR2), adhering to the “Thick Client” philosophy where React handles the UI state and Node handles the data flow.

### Drift vs. Erosion Strategy:

We embraced **Architectural Drift** (Node.js, Ollama) where it enhanced maintainability and performance. We accepted **Architectural Erosion** (TF-IDF, Limited Telemetry) only where

strict adherence to the prescription would have jeopardized the delivery timeline or added unnecessary infrastructure complexity for a single-user local tool.

## **4. Implementation Validation**

### **4.1 Key Technical Achievements**

#### **Multi-Tier Caching:**

1. IndexedDB (browser): Instant UI loads
2. Memory cache (backend): Reduces DB queries
3. MongoDB: Index-based optimization
4. Global settings: Zero-latency access

**SSE Streaming:** Ollama → Backend → Frontend with low token latency

**RAG Pipeline:** User query → `DocumentService.searchDocuments()` → TF-IDF scoring → Top-3 docs → Augmented prompt → Ollama → Cited response

**RBAC:** JWT tokens + `authMiddleware` + `adminMiddleware` enforce role-based access

### **4.2 Satisfaction of Requirements**

#### **Functional Compliance**

The implementation successfully delivers the primary user value proposition defined in the prescriptive architecture. The core chat and session management capabilities (FE1, FE2) are fully operational, with the `ChatService` and MongoDB persistence layer ensuring reliable state management across sessions. Document ingestion (FE3) is handled effectively via the `DocumentService`, enabling the required RAG workflow. However, the grounded answer capability (FE4) represents a partial satisfaction due to the documented architectural erosion; while citations are functionally present, the retrieval quality is constrained by the TF-IDF algorithm compared to the originally envisioned vector search. Administrative and developer workflows (FE5, FE6) are addressed through secure, protected routes and comprehensive repository-based documentation, ensuring distinct access control and operational visibility without the need for dedicated UI pages.

## **Non-Functional Compliance**

The system strictly adheres to the hardware and privacy constraints that drove the architectural design. By utilizing quantized 4-bit models within Ollama, the solution operates effectively within the 16GB RAM limit (NFR1) without compromising the privacy requirement (NFR5) of keeping all data and inference processing local. Latency targets (NFR2) are met through the use of React Router for optimistic UI updates and IndexedDB for client-side caching. Furthermore, the consolidation of the codebase into a unified MERN stack has demonstrably enhanced maintainability (NFR7), simplifying the development lifecycle and lowering the barrier to entry compared to the prescriptive polyglot approach.